

Politecnico di Torino

Master's Degree in Computer Engineering

Migrating Legacy Services to a Cloud-Native Architecture: A Case Study on Outsight Cloud



**Politecnico
di Torino**

Supervisor: Prof. Fulvio Giovanni Risso

Candidate: Giovanni Russo

Matricola: s343424

Academic Year 2025–2026

Abstract

This thesis addresses a concrete engineering problem: how to migrate a legacy cloud platform to a cloud-native architecture while preserving service continuity and controlling both technical and economic risk. The case study is the Oversight Cloud platform, where legacy components relied heavily on Airtable-centric data management, ad-hoc operational workflows, and infrastructure choices that limited scalability, maintainability, and long-term cost efficiency.

We define the migration problem, identify constraints, and compare candidate solutions for identity management, application hosting, storage, and data persistence. The selected direction combines managed AWS services, Infrastructure as Code with Terraform, and incremental migration from Airtable to PostgreSQL, with explicit coexistence mechanisms to reduce transition risk.

A central contribution is a migration methodology that links architectural choices to measurable criteria. Technical criteria include security, backward compatibility, observability, testability, and deployment reliability. Economic criteria include recurring infrastructure cost drivers, operational overhead, and maintainability over time. Based on these criteria, we derive a target architecture and an incremental execution plan.

The thesis documents the implemented migration foundations and the validation strategy already available in project artifacts: dual-backend smoke testing, CI pipelines for consistency checks, data replication and validation scripts, and environment-separated infrastructure definitions. The evaluation is therefore focused on migration feasibility, design quality, and evidence of risk reduction.

The main outcome is a reusable, industry-grounded framework for planning and justifying legacy-to-cloud-native migration decisions. The results are intended for practitioners and organizations that need to modernize production systems through

deliberate architectural design, not through one-off technology replacement.

Acknowledgements

I would like to express my sincere gratitude to Oversight for providing me with the opportunity to complete this internship and work on challenging and meaningful projects. I am particularly thankful to my supervisor and the entire development team for their guidance, support, and patience throughout this journey.

I would also like to thank the Politecnico di Torino for the academic foundation that made this work possible, and my thesis advisor for valuable feedback and encouragement.

Finally, I am deeply grateful to my family and friends for their unwavering support during my studies.

Contents

Abstract	I
Acknowledgements	III
List of Figures	VII
List of Tables	VIII
List of Acronyms	IX
1 Introduction	1
1.1 Industrial Context and Legacy Constraints	1
1.2 Problem Statement	1
1.3 Thesis Scope	2
1.4 Thesis Structure	2
2 Background and Related Work	5
2.1 Cloud-Native Principles for Legacy Modernization	5
2.1.1 Incremental Delivery and Operability	5
2.1.2 Infrastructure as Code	6
2.2 Migration Strategies and Patterns	6
2.2.1 Coexistence as a First-Class Requirement	6
2.2.2 Decision-Driven Migration	6
2.3 IAM Modernization in Legacy Systems	6
2.3.1 From Application-Coupled Identity to Standard Protocols . .	7
2.3.2 Open-Source and Managed IAM Trade-offs	7
2.4 Limitations in Existing Approaches	7

2.4.1	Fragmented Treatment of Technical and Economic Criteria . .	7
2.4.2	Limited Evidence on Incremental Validation	7
2.5	Positioning of This Thesis	8
3	Case Study Baseline (As-Is System)	9
3.1	Current Architecture and Data Flow	9
3.2	Operational and Cost Baseline	10
3.2.1	Operational Constraints	10
3.2.2	Economic Constraints	10
3.3	Technical Debt and Migration Drivers	10
4	Migration Methodology and Decision Framework	13
4.1	Migration Goals and Success Criteria	13
4.2	Candidate Solutions Considered	14
4.2.1	Infrastructure and Runtime Options	14
4.2.2	Data-Layer Migration Options	14
4.2.3	Identity Evolution Options	14
4.3	Technical and Economic Evaluation Criteria	14
4.3.1	Technical Criteria	14
4.3.2	Economic Criteria	15
4.4	Selected Strategy and Rationale	15
4.5	Risk Management and Coexistence Plan	15
5	Target Architecture and Solution Design (To-Be)	17
5.1	Architecture Overview	17
5.2	Identity, Data, and Storage Design	18
5.2.1	Identity Design Direction	18
5.2.2	Data-Layer Design Direction	18
5.2.3	Storage Design	18
5.3	Security, Observability, and Deployment Design	18
5.3.1	Security Design	18
5.3.2	Observability Design	19
5.3.3	Deployment Design	19
5.4	Design Decisions and Trade-offs	19

6	Migration Implementation	21
6.1	Incremental Execution Plan	21
6.2	Infrastructure as Code and Environment Separation	22
6.2.1	Terraform-Driven Provisioning	22
6.2.2	Environment Governance	22
6.3	Data Migration Tooling and Validation	22
6.3.1	Relational Foundation	22
6.3.2	Replication and Validation Scripts	22
6.4	CI/Test Strategy for Dual-Backend Consistency	23
6.4.1	Dual-Backend Test Design	23
6.4.2	Containerized Validation Environment	23
6.4.3	Pipeline Integration	23
6.5	Implementation Scope and Current Status	23
7	Evaluation and Discussion	25
7.1	Validation Methodology	25
7.2	Technical Results	25
7.2.1	Infrastructure and Deployment Readiness	25
7.2.2	Data Migration Readiness	26
7.2.3	Testing and Verification Readiness	26
7.3	Migration KPIs and Economic Impact	26
7.3.1	Technical Migration KPIs	26
7.3.2	Economic Migration KPIs	26
7.4	Limitations and Lessons Learned	27
7.4.1	Limitations	27
7.4.2	Lessons Learned	27
8	Conclusions and Future Work	29
8.1	Key Outcomes	29
8.2	Main Contributions	29
8.3	Future Improvements	30

List of Figures

List of Tables

List of Acronyms

ABAC Attribute-Based Access Control

API Application Programming Interface

CI/CD Continuous Integration / Continuous Deployment

CNCF Cloud Native Computing Foundation

IAM Identity and Access Management

IDOR Insecure Direct Object Reference

JWT JSON Web Token

OIDC OpenID Connect

RBAC Role-Based Access Control

REST Representational State Transfer

S3 Simple Storage Service (Amazon)

SAML Security Assertion Markup Language

SSO Single Sign-On

TLS Transport Layer Security

XSS Cross-Site Scripting

Chapter 1

Introduction

1.1 Industrial Context and Legacy Constraints

Many software-intensive companies operate platforms that are already cloud-hosted but not truly cloud-native in design, deployment workflow, and operational governance. In these contexts, modernization is not a greenfield exercise: it requires preserving existing behavior while progressively reducing technical debt.

This thesis focuses on Oversight Cloud, a platform used to manage LiDAR-related assets and simulations. The initial system combined a Vue frontend and a Node.js/Express backend, with a data model strongly coupled to Airtable, Redis-based sessions, containerized deployment, and reverse-proxy routing. Although functional, this setup exposed structural constraints in scalability, long-term maintainability, and cost control.

1.2 Problem Statement

The core problem addressed in this work is the migration of a legacy platform toward a cloud-native architecture under real operational constraints. The migration had to satisfy the following conditions:

- **Continuity:** avoid disruptive replacement and preserve business-critical flows during transition.
- **Security modernization:** prepare the platform for standards-based identity

and access management instead of legacy authentication assumptions.

- **Data evolution:** progressively decouple application logic from Airtable-centric persistence.
- **Operational robustness:** improve reproducibility through Infrastructure as Code, environment separation, and automated validation.
- **Economic sustainability:** justify architectural choices also in terms of recurring cost and operational effort.

1.3 Thesis Scope

The thesis focuses on designing and justifying a migration strategy for a legacy platform under real technical and economic constraints. The scope includes:

- defining the problem context and migration drivers;
- comparing candidate cloud migration options;
- selecting and motivating a target architecture;
- presenting implementation and validation foundations already supported by project evidence.

1.4 Thesis Structure

This thesis is organized as follows:

Chapter 2 introduces the conceptual and related-work foundations on cloud-native modernization, migration patterns, and IAM evolution in legacy systems.

Chapter 3 describes the case-study baseline (as-is architecture), including technical debt and cost-related migration drivers.

Chapter 4 presents the migration methodology and decision framework, including candidate options and selection rationale.

Chapter 5 defines the target architecture (to-be), its main components, and design trade-offs.

Chapter 6 details the incremental migration implementation foundations, with focus on IaC, data migration tooling, and validation pipeline.

Chapter 7 discusses the evaluation approach and current evidence regarding migration feasibility, quality, and impact.

Chapter 8 concludes the thesis by summarizing contributions and outlining future work.

Chapter 2

Background and Related Work

This chapter provides the conceptual background required to frame the migration problem and positions the thesis with respect to related work.

2.1 Cloud-Native Principles for Legacy Modernization

Cloud-native modernization is not limited to container adoption. In a migration context, it also requires reproducible infrastructure, explicit service boundaries, automated validation, and operational observability. These principles are especially relevant when modern and legacy components must coexist during transition.

2.1.1 Incremental Delivery and Operability

A recurring principle in modernization literature is incremental delivery over big-bang replacement. Incremental delivery reduces migration risk, enables early feedback, and supports rollback strategies. To be effective, it requires infrastructure automation, environment consistency, and testable interfaces between old and new components.

2.1.2 Infrastructure as Code

Infrastructure as Code (IaC) is a key enabler for modernization because it turns deployment topology, security controls, and environment configuration into versioned artifacts. This supports traceability, reviewability, and reproducibility across development and production environments.

2.2 Migration Strategies and Patterns

Common migration strategies include rehosting, replatforming, refactoring, and selective rebuilding. In practice, many enterprise projects combine these strategies across subsystems. Decision criteria generally include coupling level, criticality, migration effort, and risk exposure.

2.2.1 Coexistence as a First-Class Requirement

For systems that cannot tolerate interruption, coexistence between legacy and target components becomes a design requirement. This introduces additional complexity: contract compatibility, dual-path validation, and progressive switchover mechanisms.

2.2.2 Decision-Driven Migration

Related work often describes migration patterns conceptually, but less frequently provides explicit decision frameworks that combine technical and economic rationale. This gap is relevant for industrial contexts where architecture choices must be justified beyond purely technical preference.

2.3 IAM Modernization in Legacy Systems

Identity and Access Management (IAM) modernization is a recurrent challenge in legacy platforms. Token-based approaches and standards such as OAuth 2.0 and OpenID Connect (OIDC) provide stronger interoperability and security controls than ad-hoc authentication schemes.

2.3.1 From Application-Coupled Identity to Standard Protocols

Legacy systems frequently embed identity assumptions in application code and persistence layers. Migrating to standards-based IAM requires decoupling these assumptions, redefining authorization boundaries, and validating compatibility with existing clients and workflows.

2.3.2 Open-Source and Managed IAM Trade-offs

The choice between open-source IAM solutions and managed identity platforms affects control, integration flexibility, and operating cost. For migration projects, this decision also impacts how quickly existing services can adopt new identity flows.

2.4 Limitations in Existing Approaches

The literature provides strong foundations on migration patterns and IAM technologies, but industrial guidance is often fragmented across separate dimensions.

2.4.1 Fragmented Treatment of Technical and Economic Criteria

Technical architecture decisions are frequently discussed without detailed economic framing. However, recurring infrastructure costs, operational burden, and dependency reduction are often decisive in enterprise migration roadmaps.

2.4.2 Limited Evidence on Incremental Validation

Many case studies report final architectures but provide limited detail on intermediate validation mechanisms, such as dual-backend tests, migration data checks, and environment-separated verification workflows.

2.5 Positioning of This Thesis

This thesis addresses the above limitations through a decision-oriented migration case study. The contribution is not a generic technology survey, but an end-to-end reasoning process from baseline constraints to architecture selection, incremental implementation, and migration-focused validation.

Chapter 3

Case Study Baseline (As-Is System)

This chapter describes the initial state of the Oversight Cloud platform before the migration design was formalized. The goal is to establish the baseline against which migration decisions are evaluated.

3.1 Current Architecture and Data Flow

The as-is platform handled LiDAR-related assets and simulations through a web application and backend API stack:

- Frontend based on Vue 3.
- Backend based on Node.js/Express.
- Redis-backed session mechanisms.
- Containerized deployment with reverse-proxy routing.
- File storage through S3-compatible object storage services.
- Core business data strongly coupled to Airtable.

This architecture enabled fast delivery in earlier stages of product evolution, but it created a concentration of dependencies in external data services and manually maintained infrastructure conventions.

3.2 Operational and Cost Baseline

The baseline analysis highlighted two classes of constraints.

3.2.1 Operational Constraints

Initial platform analysis identified:

- Limited traceability in deployment and infrastructure changes.
- Residual complexity in API routing conventions.
- Difficulty in isolating and validating behavior when replacing data backends.
- Dependency on non-uniform operational knowledge across team members.

3.2.2 Economic Constraints

From the baseline reports, recurring cost and dependency concentration on Airtable emerged as significant migration drivers. Airtable was not only a persistence layer but also part of operational workflows, which increased both direct cost exposure and long-term coupling risk.

3.3 Technical Debt and Migration Drivers

The migration need was not framed as a generic technology update, but as a response to specific debt patterns:

- **Data-layer coupling:** application behavior strongly tied to Airtable schema and workflows.
- **Identity evolution pressure:** growing need for standards-based authentication and cleaner authorization boundaries.
- **Infrastructure reproducibility gaps:** insufficiently formalized deployment model for multi-environment evolution.
- **Validation limitations:** absence of a robust incremental strategy to verify parity during backend transition.

These drivers motivated a migration program with two design priorities: incremental risk reduction and explicit decision traceability.

Chapter 4

Migration Methodology and Decision Framework

This chapter formalizes the migration methodology used in this thesis. The emphasis is on how decisions were made, not only on which technologies were selected.

4.1 Migration Goals and Success Criteria

The migration goals were defined before implementation to avoid solution bias:

- Preserve operational continuity during transition.
- Reduce architectural coupling to legacy data and identity assumptions.
- Improve deployment reproducibility and environment governance.
- Provide explicit technical and economic justification for major choices.

From these goals, the following success criteria were derived:

- ability to run and validate critical flows during data-layer transition;
- traceable infrastructure definitions across environments;
- measurable reduction of dependency concentration on Airtable;
- migration process auditable through tests, scripts, and CI evidence.

4.2 Candidate Solutions Considered

The decision process considered multiple options for each migration dimension.

4.2.1 Infrastructure and Runtime Options

- Keep existing deployment style with incremental hardening.
- Move to managed cloud services with IaC governance.
- Adopt a full orchestration-first redesign from the beginning.

4.2.2 Data-Layer Migration Options

- Immediate cutover from Airtable to a relational database.
- Long-term coexistence with no formal migration endpoint.
- Incremental coexistence with progressive parity checks and controlled switchover.

4.2.3 Identity Evolution Options

- Preserve legacy authentication contracts and postpone IAM modernization.
- Introduce standards-based IAM with progressive integration in services.
- Full immediate rewrite of authentication and authorization boundaries.

4.3 Technical and Economic Evaluation Criteria

Each candidate solution was evaluated against a shared set of criteria.

4.3.1 Technical Criteria

- Backward compatibility with existing service behavior.
- Security posture and readiness for standards-based IAM.
- Deployment reproducibility across environments.

- Testability of migration steps and rollback readiness.
- Observability and troubleshooting support during transition.

4.3.2 Economic Criteria

- Recurring infrastructure and platform costs.
- Operational overhead for day-2 activities.
- Effort required for migration execution and maintenance.
- Risk of long-term vendor or tooling dependency concentration.

4.4 Selected Strategy and Rationale

The selected strategy was an incremental migration centered on managed cloud services and reproducible infrastructure definitions:

- AWS-managed building blocks for runtime, routing, storage, and observability.
- Terraform as the infrastructure control plane.
- Progressive data migration path from Airtable to PostgreSQL.
- Validation-oriented coexistence between legacy and target data paths.

This strategy was selected because it balances risk, implementation effort, and long-term maintainability. It avoids disruptive full replacement while still creating a clear target-state trajectory.

4.5 Risk Management and Coexistence Plan

Risk management was embedded in the migration design:

- Keep migration steps independently verifiable.
- Separate development and production environment evolution.

- Use dual-backend test paths to detect behavioral divergence early.
- Define data validation artifacts to support switchover decisions.

The coexistence phase is treated as a controlled engineering stage, not as an informal transition period.

Chapter 5

Target Architecture and Solution Design (To-Be)

This chapter presents the target architecture resulting from the decision framework in Chapter 4. The architecture is described as a migration destination designed for incremental adoption.

5.1 Architecture Overview

The target architecture is organized around four concerns:

- **Application runtime:** containerized frontend and backend services deployed on managed cloud compute.
- **Traffic and exposure:** explicit HTTP routing and health endpoints through managed load balancing.
- **Persistence and assets:** relational persistence for business data and object storage for simulation files.
- **Operations:** IaC-controlled provisioning, log collection, and environment-specific governance.

5.2 Identity, Data, and Storage Design

5.2.1 Identity Design Direction

The target identity direction is standards-based and service-oriented. Authentication and authorization concerns are progressively decoupled from application-specific legacy assumptions, enabling future CIAM/SSO integration while preserving migration continuity.

5.2.2 Data-Layer Design Direction

The data architecture moves from Airtable-centric coupling toward PostgreSQL-based persistence under explicit schema control. Migration is designed as incremental coexistence, supported by replication and validation tooling.

5.2.3 Storage Design

Simulation and file assets are maintained in private object storage with CDN distribution where required. Access patterns are controlled through IAM-based permissions and bounded service responsibilities.

5.3 Security, Observability, and Deployment Design

5.3.1 Security Design

Security decisions prioritize:

- least-privilege access policies for services and automation;
- secrets handling through managed secret services;
- explicit separation between public entry points and internal service surfaces.

5.3.2 Observability Design

The target architecture includes centralized operational visibility through managed logging and monitoring services. This is essential for migration because it provides evidence during coexistence and switchover phases.

5.3.3 Deployment Design

Deployment design is Terraform-driven, with explicit environment separation and reviewable infrastructure changes. This addresses reproducibility and reduces configuration drift across stages.

5.4 Design Decisions and Trade-offs

The final architecture reflects trade-offs rather than absolute optimization:

- managed services reduce platform maintenance effort but introduce configuration discipline requirements;
- incremental migration reduces cutover risk but increases temporary system complexity;
- coexistence validation improves confidence but requires dedicated testing and data checks.

Overall, the chosen design prioritizes migration safety and long-term maintainability over short-term simplification.

Chapter 6

Migration Implementation

This chapter describes the implementation foundations that were already established to execute the migration strategy in a controlled and verifiable way.

6.1 Incremental Execution Plan

The migration was implemented incrementally, with explicit separation between foundation work and full functional cutover. The implementation sequence followed four tracks:

- establish cloud infrastructure foundations on AWS;
- formalize reproducible provisioning with Terraform;
- introduce data migration tooling from Airtable to PostgreSQL;
- implement validation pipelines to check behavioral consistency during coexistence.

This sequencing reduced migration risk by enabling technical validation before irreversible system decisions.

6.2 Infrastructure as Code and Environment Separation

6.2.1 Terraform-Driven Provisioning

Infrastructure provisioning was implemented using Terraform, with explicit definitions for networking, routing, compute, storage, and observability-related services. The resulting setup includes managed components such as load balancing, container runtime services, DNS/TLS integration, and cloud logging.

6.2.2 Environment Governance

A dedicated effort was made to separate development and production concerns. This included environment-specific configuration handling and controlled evolution of shared resources to avoid production side effects during migration experimentation.

6.3 Data Migration Tooling and Validation

6.3.1 Relational Foundation

The migration path from Airtable to PostgreSQL was implemented with schema and migration tooling designed to preserve compatibility constraints. The objective was not only data transfer, but controlled evolution toward explicit schema ownership.

6.3.2 Replication and Validation Scripts

Dedicated scripts were prepared to:

- replicate data from Airtable into PostgreSQL;
- validate consistency through machine-readable reports;
- support migration checkpoints before wider rollout.

These artifacts transform data migration from a one-time operation into a repeatable verification workflow.

6.4 CI/Test Strategy for Dual-Backend Consistency

6.4.1 Dual-Backend Test Design

To support coexistence, smoke tests were designed to run against both data backends (Airtable and PostgreSQL) under a common API expectation. This provided an early warning mechanism for behavioral divergence.

6.4.2 Containerized Validation Environment

The test workflow uses containerized dependencies, including a PostgreSQL service and an Airtable-compatible mock component, so that migration checks can be reproduced reliably in CI.

6.4.3 Pipeline Integration

GitLab CI stages were defined to execute unit and smoke checks across backend variants. This allows migration decisions to be supported by continuous evidence instead of manual spot checks.

6.5 Implementation Scope and Current Status

At the current stage, the implemented work establishes the migration backbone:

- infrastructure foundations and environment governance;
- data replication and validation tooling;
- coexistence-oriented CI/test pipeline.

This scope is sufficient to evaluate migration feasibility and design quality, while leaving room for further functional expansion in subsequent project phases.

Chapter 7

Evaluation and Discussion

This chapter evaluates the migration approach with focus on decision quality, implementation readiness, and risk reduction evidence.

7.1 Validation Methodology

Given the current project stage, evaluation is organized around migration readiness rather than final production performance optimization. The methodology combines:

- architectural consistency checks between migration goals and implemented foundations;
- reproducibility checks through Infrastructure as Code and environment separation;
- coexistence checks through dual-backend testing and CI evidence;
- data migration checks through replication and validation scripts.

7.2 Technical Results

7.2.1 Infrastructure and Deployment Readiness

The migration produced a reproducible infrastructure baseline on AWS, managed through Terraform and integrated with routing, certificate management, container runtime, and observability services.

7.2.2 Data Migration Readiness

The project established concrete mechanisms to compare and validate behavior during Airtable-to-PostgreSQL transition. This is a key technical result because it enables incremental migration decisions based on evidence rather than assumptions.

7.2.3 Testing and Verification Readiness

Dual-backend smoke tests and CI integration provide an automated control surface for migration risk. This increases confidence in backward-compatible evolution during coexistence phases.

7.3 Migration KPIs and Economic Impact

The evaluation framework tracks migration progress using two KPI classes.

7.3.1 Technical Migration KPIs

- percentage of critical flows covered by backend-agnostic smoke tests;
- number of migration checkpoints completed with successful validation reports;
- reproducibility of environment provisioning across development stages;
- incidence of regressions detected before deployment.

7.3.2 Economic Migration KPIs

- reduction in dependency concentration on high-recurring-cost services;
- expected decrease in manual operational effort through IaC and pipeline automation;
- migration effort distribution across infrastructure, data, and application layers.

Initial evidence confirms that the selected strategy is compatible with both technical risk control and economic sustainability goals.

7.4 Limitations and Lessons Learned

7.4.1 Limitations

The current evaluation reflects an intermediate migration stage. Long-term operational behavior and full post-migration cost curves require additional observation after broader adoption.

7.4.2 Lessons Learned

Three lessons emerge clearly:

- migration architecture must be paired with explicit validation architecture;
- coexistence is manageable only when test and data checks are planned from the start;
- technical and economic criteria must be evaluated together, not sequentially.

Chapter 8

Conclusions and Future Work

This thesis addressed the migration of a legacy platform toward a cloud-native architecture as a structured engineering decision problem. The work focused on defining and justifying a migration strategy under technical and economic constraints.

8.1 Key Outcomes

The migration approach demonstrates that legacy-to-cloud-native transition can preserve continuity when it is treated as an incremental engineering process with explicit validation checkpoints.

The decision framework confirms the need to evaluate technical and economic criteria together. Security, compatibility, and operability decisions are sustainable only when recurring cost and operational effort are considered from the beginning.

The implementation evidence shows that coexistence can be managed through dual-backend testing, CI-enforced checks, and data validation artifacts. These elements make migration progress measurable and auditable.

Finally, the produced artifacts (decision rationale, Terraform definitions, migration scripts, validation workflows) improve reusability of the approach beyond a single project context.

8.2 Main Contributions

The main contributions of this thesis are:

- a baseline analysis that makes migration drivers explicit;
- a migration methodology and decision framework grounded in technical and economic evaluation;
- a target architecture aligned with incremental migration constraints;
- an implementation backbone for controlled execution and evidence-based validation.

8.3 Future Improvements

Future work can extend this thesis in three directions:

- complete and monitor full production cutover stages with long-term KPIs;
- further evolve identity architecture toward full standards-based integration across all services;
- consolidate post-migration cost and reliability analysis with extended operational data.

Overall, the thesis shows that cloud-native migration is most effective when treated as an explicit design process supported by verifiable checkpoints, rather than as a sequence of isolated implementation tasks.